

Half of Your CMDB's Measured Value Is the Routing Queue You Already Have

Sudhir Dixit

Independent Researcher
sudhir.dixit1@gmail.com

Abstract

Configuration management database (CMDB) programmes are justified by the analytics they enable, and that justification is a comparison against a baseline the analyst chooses. On a public event log of 45,455 incidents from a bank's IT operation whose affected-item field is 100% populated, we show the choice is worth as much as the data. Predicting misrouting at incident creation, knowing the affected configuration item is worth +0.183 AUC against four intake fields, and +0.103 [+0.094, +0.114] once the opening assignment queue is added — a field every service desk records at intake, for free. Two measurements explain the gap. Adding the queue to a model that already knows the item is worth under 0.01 AUC. And randomising the queue label within each item, which destroys the queue's identity but keeps what the item implies about it, still retains 91% of the queue's own gain against a matched floor of 2%: the routing decision is very nearly a function of the affected item. We report two attempts that failed — a third free-looking field drives the measured gain to zero and we could not establish whether it is admissible, and a reverse-direction measurement whose defensible nulls disagree.

1 Introduction

A CMDB programme is expensive and is justified by what it will enable. That justification rests on a comparison whose second arm — performance without configuration data — is a choice the analyst makes, usually silently, by deciding which already-available fields to include.

On one organisation's incident log the choice halves the answer, and we can say exactly where the difference goes. This is a short paper with one result and one mechanism.

On what this paper used to claim. Earlier versions of this work reported several larger findings that were withdrawn under adversarial review, each an artifact of a control we had constructed. The paper is deliberately smaller as a result: every claim below is either a direct measurement or is reported against a null, and where a null is ambiguous we say so and drop the claim.

2 Background

The CMDB and what justifies it. A CMDB records configuration items and their relationships, and is the asset on which the configuration-management practice of ITIL and its successors rests. Survey work on ITSM framework adoption (Marrone and Kolbe 2011) finds that realised benefits grow with implementation maturity, but measures them as practitioner-reported outcomes rather than as the analytic value of the configuration data itself. We are not aware of a peer-reviewed measurement of what knowing the affected configuration item is worth to a named prediction task; the material addressing that question directly is vendor documentation. The gap is the occasion for this paper.

The methods a CMDB is bought to enable. Surveys of AIOps for failure management (Zhang et al. 2024; Remil et al. 2024; Notaro, Cardoso, and Gerndt 2021) catalogue triage, failure-prediction and root-cause methods that take configuration data as input. Incident triage in particular — deciding which team owns an incident — has been treated as a learning problem on operational logs at industrial scale (Chen et al. 2019), as has failure prediction across a cloud estate (Lin et al. 2018).

Prediction on incident event logs. Our task sits in predictive process monitoring, where outcome prediction is surveyed and benchmarked across real-world logs (Teinemaa et al. 2019; Verenich et al. 2019) on data of the kind process mining formalises (van der Aalst 2016). The BPI Challenge logs are the standard public instances (van Dongen 2014; Steeman 2013) and comparable ITSM exports exist elsewhere (Amaral, Fantinato, and Peres 2018). Such logs carry documented quality hazards: the imperfection patterns catalogued by Suriadi et al. (2017) include several we rule out explicitly below, in particular values denormalised from a record onto its events.

What the comparison is measured against. Our question is not whether those methods work but what their evaluation is measured against. Rendle, Zhang, and Koren (2019) show for recommender systems that inadequately specified baselines can account for reported gains, and Ferrari Dacrema, Cremonesi, and Jannach (2019) report the same pattern across a body of neural recommendation work. The concern generalises: benchmark conclusions are sensitive to sources

of variation that are seldom reported (Bouthillier et al. 2021), and documentation of the choices producing a result is thin across AI research (Gundersen and Kjensmo 2018). Where the choice concerns which fields may enter a model it shades into leakage, whose formulation and detection are treated by Kaufman et al. (2012) and whose consequences across empirical fields are catalogued by Kapoor and Narayanan (2023); for predictive process monitoring specifically, Weytjens and De Weerd (2022) show that benchmark datasets need explicit construction to avoid it. We report this failure mode for a deployment decision rather than a leaderboard, where the field-admission choice is not a modelling detail but the number the business case turns on.

3 Data and Task

We use the incident records of the BPI Challenge 2014 log (van Dongen 2014) from Rabobank Group ICT, recorded in HP Service Manager, with the activity log shipped alongside.

Cleaning. Of 46,809 rows, 203 carry no incident identifier and one an unparseable timestamp. A further 1,150 incidents opened before 2013-10-01 are left-censored — already open when the extract began — and show reassignment rates of 76–100% against a subsequent 35–42% that declines gently across the window. Removing them leaves 45,455 incidents to 2014-03-31; a temporal 70/30 split gives 31,818 training and 13,637 test incidents, positive rates 0.413 and 0.372. The cutoff is not taken from the outcome: monthly volume rises from 857 incidents in September 2013 to 8,606 in October. Over the analysed rows the affected-item field is 100% populated across 2,929 items, 2,554 of them seen in training.

Task and estimator. Predict at creation whether an incident will later be reassigned. Median handling time rises from 1.59 hours for incidents never reassigned to 47.53 hours for those reassigned five or more times; we use this to motivate the task, not as a recoverable saving. The estimator is one-hot logistic regression, chosen because it is bin-free. Encoders and rankings are fit on training data only; intervals are 2,000-resample paired bootstraps.

The opening routing queue. The activity log records, on the Open event of every incident, the queue the service desk routed it to: 50 groups in the analysed cohort, none missing. The Open activity is the earliest activity for all 45,455 incidents, its timestamp never precedes the record’s open time, and the value varies within an incident for 92.56% of cases in the cohort — so it is a genuine per-event observation available at creation, not a closing state. It is also not the resolving queue: it matches the last-observed queue for only 21.35% of incidents. We note that an early version of this work excluded assignment group outright as a handling-time field. That ruling was about the queue recorded at closure; the value on the opening row is a different variable, and the checks above are why we admit it.

Field mutation, disclosed. Because the incident file is a closed-record snapshot, we report how often each field we use is edited after creation: Urgency on 1,107 incidents (2.44%), Impact on 1,084 (2.38%), the affected item on 159 (0.35%).

Baseline	AUC	+ item id.	gain [95% CI]
intake only	0.562	0.746	+0.183 [+0.172, +0.195]
+ routing queue	0.644	0.748	+0.103 [+0.094, +0.114]

Table 1: The value of knowing the affected configuration item, with and without a field the service desk already records.

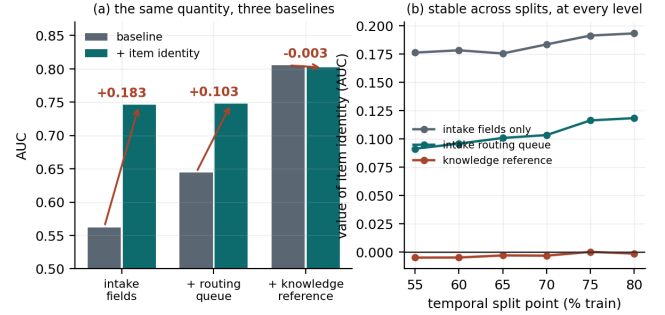


Figure 1: (a) The same quantity under three baselines; the third is discussed in Section 6. (b) The ordering holds at every temporal split point from 55% to 80%.

Incidents whose Impact or Urgency was edited are more likely to be reassigned (0.66 against 0.39), so this is not neutral; Section 4 reports the headline with those fields dropped and with the edited incidents removed.

4 The Result

Against four intake fields the affected item looks decisive. Adding the opening queue cuts its measured value by 44%. The gains are 28.1 and 17.4 standard deviations outside a null in which the incident-to-item association is shuffled, preserving column count and marginal distribution while destroying the signal — a comparison that matters because adding item identity adds 2,554 indicator columns, which is not free under a fixed penalty. These z figures pool the null’s Monte-Carlo dispersion with the gain’s own bootstrap standard error; dividing by the null alone, as an earlier draft did, would report 51 and 33.

The result is not a split artifact: across six temporal split points the two gains range +0.176 to +0.193 and +0.091 to +0.118 (Figure 1b). The bootstrap interval conditions on every design choice, so it is narrower than the honest spread: across split point, target threshold and cleaning cutoff the second gain ranges +0.068 to +0.130, and it rises with training volume. Nor does the disclosed field editing explain it: reducing the intake block to Category alone — dropping Impact and Urgency, which are edited, and Priority, which is a deterministic function of them — gives +0.106, and restricting to the 44,227 never-edited incidents gives +0.107.

5 Where the Difference Goes

The two rows differ by 0.080. That number is not itself informative — for any two variables, the drop in one’s measured gain when the other joins the baseline equals the drop in the

Quantity	AUC
item’s gain over intake, $A_i - A_0$	+0.183
item’s gain once the queue is present, $A_{qi} - A_q$	+0.103
queue’s gain over intake, $A_q - A_0$	+0.082
queue’s gain once the item is present, $A_{qi} - A_i$	+0.002

Table 2: The overlap, read four ways. The final row is the one that matters.

other’s, so it restates the table rather than explaining it. Two direct measurements do explain it.

First: adding the routing queue to a model that already knows the affected item is worth +0.0017 [+0.0001, +0.0034]. It is positive, and it clears a matched-dimension null of -0.0009 ± 0.0006 that no draw reaches — so it is not the cost of adding columns. But across split points and penalties it ranges +0.0002 to +0.0072, so the honest statement is that the queue’s unique contribution is *under 0.01 AUC and not resolvable more finely*. Whatever the queue contributes, the item column has almost entirely absorbed it.

Second, and more directly: the queue is nearly a function of the item. Randomising the queue label *within* each item — which destroys the queue’s own identity while preserving what the item implies about it — still retains 91% of the queue’s +0.082 gain. The matched floor, the same randomisation within cells drawn to reproduce the item mass profile, retains 2%. The margin is 89 points. Knowing which item an incident concerns is very nearly enough to reconstruct the desk’s routing decision.

What we do not claim. The reverse framing — that the item column “stands in for” the queue — is not established. We measured it, by randomising items within queues, and obtained 44% of the item’s gain; but a random 50-cell grouping of items that knows nothing about routing retains 25%, and a grouping matched on the queue’s mass profile retains 7%. Defensible nulls disagree about the floor, so the number is not interpretable and we exclude it. The two measurements above are what the data supports.

6 One Thing We Could Not Establish

A third field, the knowledge-article reference, sits on the same Open row, is 100% populated, and costs nothing. Adding it to the baseline raises AUC to 0.805 and drives the measured value of item identity to -0.003 . Whether that figure belongs in Table 1 depends on whether the field is available at creation, and we could not determine this.

We tried two structural tests and report both failures. A field that varies within an incident is certainly a per-event observation; the knowledge reference never varies. But neither does `Interaction ID`, which has 45,426 distinct values for 45,455 incidents and is plainly a creation-time key — so constancy is evidence of granularity, not of timing. We then tried exact identity with the closed record: the knowledge reference matches its closed-record column for 100.000000% of incidents. But restricted to the 42,151 incidents with exactly one related interaction, `Interaction ID` matches

its own closed-record column for 99.997628% — one mismatch — against a pass mark of 100.000000%. The second test cannot separate its own counterexample either.

We therefore make no claim about this field. The -0.003 is additionally inside the dimensionality null described in Section 4, and changes sign under per-condition penalty tuning and at other split points, so it is not resolvable here on either ground. We report the attempt because the negative is the transferable part: a field being free, populated and present on an opening record does not make it creation-time, and we know of no test on this export that settles it.

7 Estate Concentration

Required CMDB scope is bounded by a property of the estate that needs no model. Here the top 8 items generate 30.2% of incidents, the top 64 generate 70.5%, and the top 128 — 5.0% of the training vocabulary — generate 82.0%. An organisation can compute this from a frequency count before funding anything. Identifying only those items recovers most of the measured value: the top 8 recover 57.9% of the +0.103, the top 64 recover 90.0%, and the top 128 recover 95.4%. We make no claim here about how much of the identity gain a given coverage recovers; an earlier draft did, from a comparison in which one of the three selection rules had no observations at the coverage levels where the claim was made.

8 Limitations

One organisation, one platform, one task, six months, one estimator family. The values +0.183 and +0.103 are properties of this estate; the transferable claims are that the gap exists, that it is large, and that the mechanism is the item column proxying for the queue. Reassignment is a proxy for misrouting and also fires on legitimate escalation; the magnitude is threshold-dependent (requiring two or more reassignments gives +0.068) though the ordering is not. The within-queue shuffle preserves queue membership exactly but not any other correlate of item identity, so it bounds the proxy share rather than isolating it perfectly.

9 Conclusion

Knowing which configuration item an incident concerns is worth +0.183 or +0.103 AUC for predicting misrouting, depending on whether the baseline includes a routing queue the service desk already records — and the difference is an overlap: the queue is very nearly a function of the item, so a model given the item has already been told the queue. A business case built on the first figure is not measuring a CMDB. It is measuring a CMDB plus a decision the organisation had already made for free.

Acknowledgements

This work benefited substantially from the review and domain challenge of a practitioner working in IT service management. Several of the analyses withdrawn from earlier versions were withdrawn because of objections raised in that review.

Reproducibility

The dataset is public (van Dongen 2014). Analysis and figure code, and a checker that compares each quantity here against a generated result file, are available at <https://github.com/sudhirdixit1/cmd-db-routing-queue-baseline>.

References

- Amaral, C.; Fantinato, M.; and Peres, S. M. 2018. Incident Management Process Enriched Event Log. UCI Machine Learning Repository. Extracted from the audit system of a ServiceNow instance; 141,712 events over 24,918 incidents. CC BY 4.0.
- Bouthillier, X.; Delaunay, P.; Bronzi, M.; Trofimov, A.; Nichyporuk, B.; Szeto, J.; Mohammadi Sepahvand, N.; Raff, E.; Madan, K.; Voleti, V.; Ebrahimi Kahou, S.; Michalski, V.; Arbel, T.; Pal, C.; Varoquaux, G.; and Vincent, P. 2021. Accounting for Variance in Machine Learning Benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*.
- Chen, J.; He, X.; Lin, Q.; Zhang, H.; Hao, D.; Gao, F.; Xu, Z.; Dang, Y.; and Zhang, D. 2019. Continuous Incident Triage for Large-Scale Online Service Systems. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- Ferrari Dacrema, M.; Cremonesi, P.; and Jannach, D. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*.
- Gundersen, O. E.; and Kjensmo, S. 2018. State of the Art: Reproducibility in Artificial Intelligence. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*.
- Kapoor, S.; and Narayanan, A. 2023. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns*, 4(9).
- Kaufman, S.; Rosset, S.; Perlich, C.; and Stitelman, O. 2012. Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4).
- Lin, Q.; Hsieh, K.; Dang, Y.; Zhang, H.; Sui, K.; Xu, Y.; Lou, J.-G.; Li, C.; Wu, Y.; Yao, R.; Chintalapati, M.; and Zhang, D. 2018. Predicting Node Failure in Cloud Service Systems. In *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- Marrone, M.; and Kolbe, L. M. 2011. Impact of IT Service Management Frameworks on the IT Organization: An Empirical Study on Benefits, Challenges and Processes. *Business & Information Systems Engineering*, 3(1): 5–18.
- Notaro, P.; Cardoso, J.; and Gerndt, M. 2021. A Survey of AIOps Methods for Failure Management. *ACM Transactions on Intelligent Systems and Technology*, 12(6).
- Remil, Y.; Bendimerad, A.; Mathonat, R.; and Kaytoue, M. 2024. AIOps Solutions for Incident Management: Technical Guidelines and A Comprehensive Literature Review. *arXiv preprint arXiv:2404.01363*.
- Rendle, S.; Zhang, L.; and Koren, Y. 2019. On the Difficulty of Evaluating Baselines: A Study on Recommender Systems. *arXiv preprint arXiv:1905.01395*.
- Steeman, W. 2013. BPI Challenge 2013, Incidents. 4TU.ResearchData. Volvo IT incident and problem management, VINST system.
- Suriadi, S.; Andrews, R.; ter Hofstede, A. H. M.; and Wynn, M. T. 2017. Event Log Imperfection Patterns for Process Mining: Towards a Systematic Approach to Cleaning Event Logs. *Information Systems*, 64: 132–150.
- Teinemaa, I.; Dumas, M.; La Rosa, M.; and Maggi, F. M. 2019. Outcome-Oriented Predictive Process Monitoring: Review and Benchmark. *ACM Transactions on Knowledge Discovery from Data*, 13(2): 17:1–17:57.
- van der Aalst, W. M. P. 2016. *Process Mining: Data Science in Action*. Springer, 2nd edition.
- van Dongen, B. F. 2014. BPI Challenge 2014. 4TU.ResearchData. Rabobank Group ICT, HP Service Manager; incident, change and interaction details.
- Verenich, I.; Dumas, M.; La Rosa, M.; Maggi, F. M.; and Teinemaa, I. 2019. Survey and Cross-Benchmark Comparison of Remaining Time Prediction Methods in Business Process Monitoring. *ACM Transactions on Intelligent Systems and Technology*.
- Weytjens, H.; and De Weerd, J. 2022. Creating Unbiased Public Benchmark Datasets with Data Leakage Prevention for Predictive Process Monitoring. In *Business Process Management Workshops*, volume 436 of *Lecture Notes in Business Information Processing*, 18–29. Springer.
- Zhang, L.; Jia, T.; Jia, M.; Wu, Y.; Liu, A.; Yang, Y.; Wu, Z.; Hu, X.; Yu, P. S.; and Li, Y. 2024. A Survey of AIOps for Failure Management in the Era of Large Language Models. *arXiv preprint arXiv:2406.11213*.